# ENCE260: Embedded Systems

## Lecture 7: Serial Communications

How to make chips talk to each other.

# Revision

An embedded system using round-robin is running:

- A $20$ Hz task that takes $20$ ms;
- A $100$ Hz tasks that takes $3$ ms.

What is the CPU load?

Does changing the scheduler used change your answer?

# Revision

Since $100\,\text{Hz}$ is a multiple of $20\,\text{Hz}$, $T_p$ is the period of the slower task.

$T_p = \frac{1}{20\,\text{Hz}} = 50\,\text{ms}.$

$T_1 = 20\,\text{ms}.$

$T_0 = 3\,\text{ms} \times \frac{100\,\text{Hz}}{20\,\text{Hz}} = 3 \times 5 = 15\,\text{ms}.$

$\sum_{i=1}^{N} T_i = T_1 + T_2 = 35\,\text{ms}.$

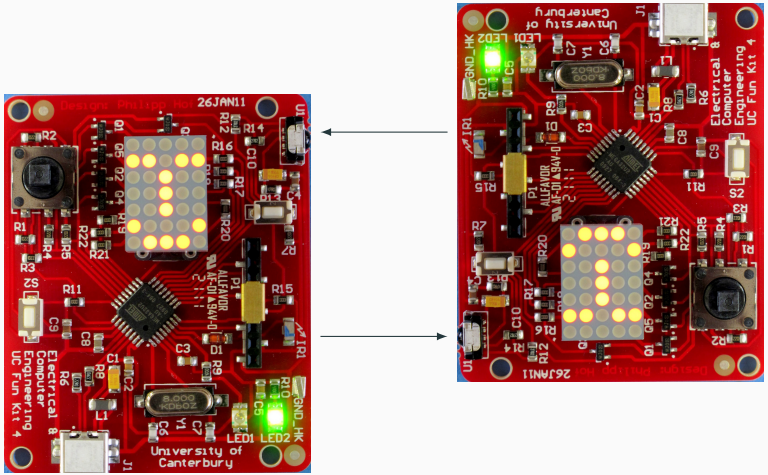
Load = $\frac{T_1 + T_2}{T_p} = \frac{35\,\text{ms}}{50\,\text{ms}} = 70\,\%.$

CPU load only cares about task duration, so scheduler choice doesn't matter.

# Inter-Chip Communication

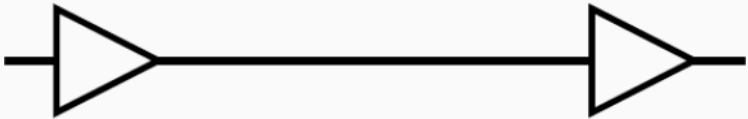
- Often multiple ICs need to communicate data between each other.
- This is true both for chips on the same board, and chips on different boards.
- When we send bits of data one after the other, we call it *serial* communication (in contrast with *parallel* communication).
- Even if over a single wire/channel, there are many details to pin down:
  - Bit polarity (does a high or low indicate 1 or 0?)
  - Bit order (do we send the MSB or LSB first?)
  - Message encoding (are we using ASCII? Unicode? Something else?)
  - Bit synchronisation (when does each bit start?)

# Communication Between Two UCFKs



# Simplex Communication

- Only one channel is present.
- Data can only be sent in one direction.
- E.g. TV, radio, any other broadcast...

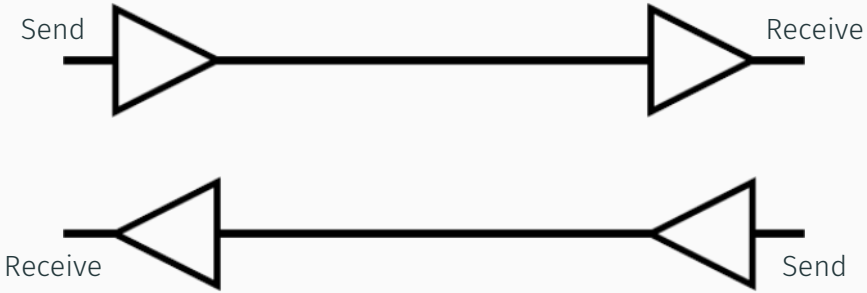


Send

Receive

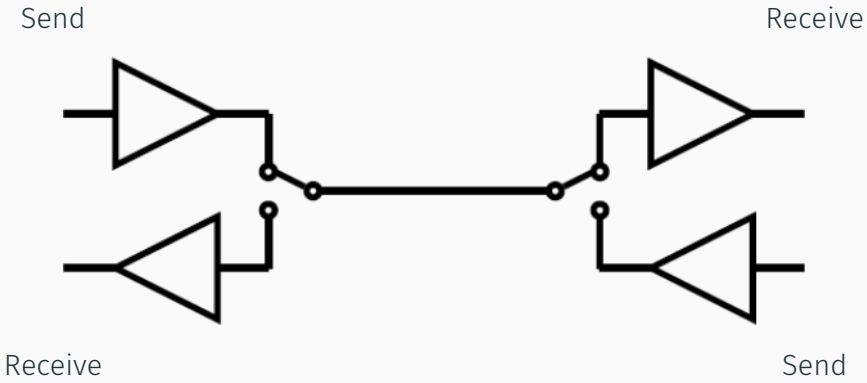
# Full Duplex Communication

- Works with one or two channels (depending on medium).
- Data can be sent in both directions simultaneously.
- E.g. phone call, Fun Kit (technically).



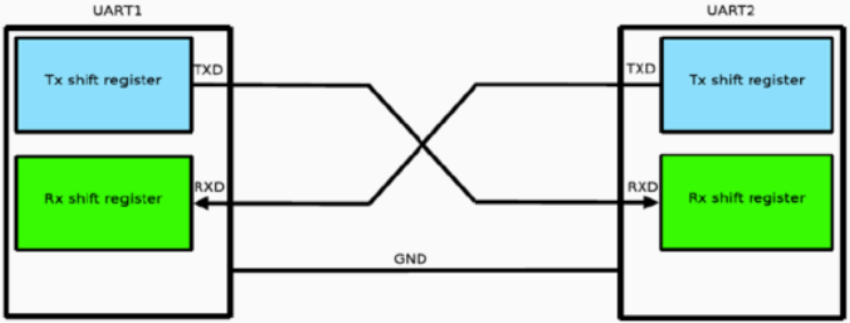
# Half Duplex Communication

- Works with a single channel
- Data can be sent in both directions, but only in a single direction at a time.
- E.g. walkie-talkie.



# UART

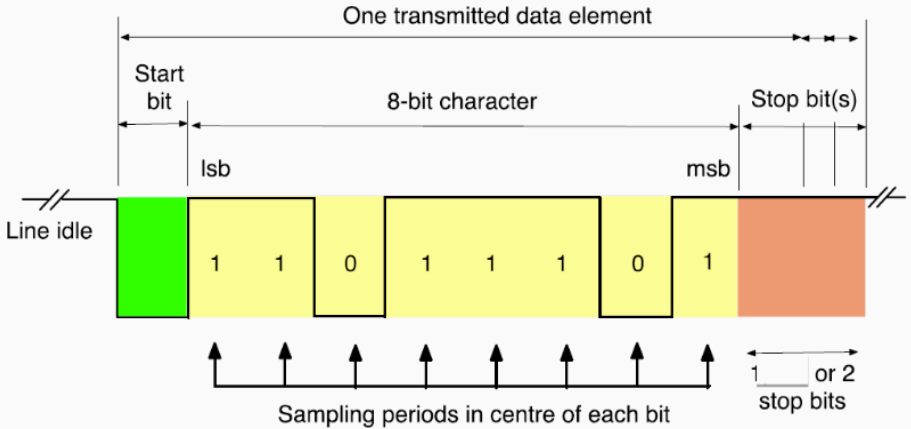
- Universal Asynchronous Receiver/Transmitter.
- Commonly used form of serial communications (i.e. bit by bit) for micros.
- One wire/channel per direction (plus common ground). ∴ **Full duplex**.
- Asynchronous (Tx and Rx need to use pre-coordinated clock frequencies).



# Baud Rate

- Since transmission is serial, the transmission speed is given in the maximum number of bits per second.
- Known as the Baud rate (max number of bit transitions per second).
- Units are “baud” or “bit/s”.
- We need a *start bit* to indicate the start of the frame (also synchronises the receiver).
- We need *stop bit*(s) to indicate the end of the frame.
- We may also use a *parity bit* for error checking.
- Odd parity means there are an odd number of 1s in the data and parity bits.
- Even parity means there are an even number of 1s in the data and parity bits.

# Serial Data Transmission




- Typically transmit LSB first.
- Optional parity bit before stop bits.

# ASCII Transmission

- American Standard Code for Information Interchange.
- Only specifies the meaning of a 7-bit unsigned integer, 8th bit is often ignored.
- $2^7 = 128$ possible characters. Values 0 - 127.
- We don't *have* to use ASCII, but it does makes things human-readable.
- The Fun Kit sends whole 8-bit bytes. Sending ASCII characters will not make use of the MSB.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | NUL | 32 | | 64 | @ | 96 | ‘ |
| 1 | ^A | 33 | ! | 65 | A | 97 | a |
| 2 | STX | 34 | " | 66 | B | 98 | b |
| 3 | ETX | 35 | # | 67 | C | 99 | c |
| 4 | ^D | 36 | $ | 68 | D | 100 | d |
| 5 | ^E | 37 | % | 69 | E | 101 | e |
| 6 | ^F | 38 | & | 70 | F | 102 | f |
| 7 | BELL | 39 | ’ | 71 | G | 103 | g |
| 8 | BS | 40 | ( | 72 | H | 104 | h |
| 9 | TAB | 41 | ) | 73 | I | 105 | i |
| 10 | LF | 42 | * | 74 | J | 106 | z |
| 11 | VT | 43 | + | 75 | K | 107 | k |
| 12 | FF | 44 | , | 76 | L | 108 | l |
| 13 | CR | 45 | - | 77 | M | 109 | m |
| 14 | ^N | 46 | . | 78 | N | 110 | n |
| 15 | ^O | 47 | / | 79 | O | 111 | o |
| 16 | ^P | 48 | 0 | 80 | P | 112 | p |
| 17 | XON | 49 | 1 | 81 | Q | 113 | q |
| 18 | ^R | 50 | 2 | 82 | R | 114 | r |
| 19 | XOFF | 51 | 3 | 83 | S | 115 | s |
| 20 | ^T | 52 | 4 | 84 | T | 116 | t |
| 21 | ^U | 53 | 5 | 85 | U | 117 | u |
| 22 | ^V | 54 | 6 | 86 | V | 118 | v |
| 23 | ^W | 55 | 7 | 87 | W | 119 | w |
| 24 | ^X | 56 | 8 | 88 | X | 120 | x |
| 25 | ^Y | 57 | 9 | 89 | Y | 121 | y |
| 26 | ^Z | 58 | : | 90 | Z | 122 | z |
| 27 | ESC | 59 | ; | 91 | [ | 123 | { |
| 28 | FS | 60 | < | 92 | \ | 124 | \| |
| 29 | GS | 61 | = | 93 | ] | 125 | } |
| 30 | RS | 62 | > | 94 | ^ | 126 | ~ |
| 31 | US | 63 | ? | 95 | _ | 127 | DEL |

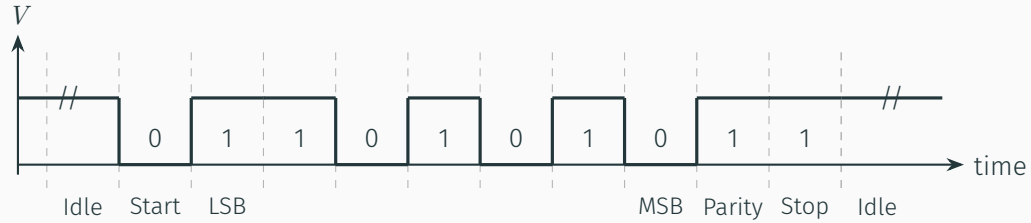
# Example Transmission

- Transmit the ASCII character for `'+'` , assuming LSB first 7-bit data, 1 start bit, 1 stop bit, and an odd parity bit.

  `'+'` has a decimal value of 43.

  Binary is: 0b 010 1011

  There are four 1s in the data, so parity bit = 1 for odd parity.



# USART Registers

- On the ATmega32U2 the UART peripheral is called `USART1`, since it can also perform synchronous communications.
- The USART registers are:
  - Control and Status Register (`UCSR1A`), which shows when the transmit data register is empty and when the receive data register has a new value.
  - Data Register (`UDR1`), which when written places the data in the Transmit Data Buffer Register, and when read returns the Receive Data Buffer Register.
  - Baud Rate Register (`UBRR`), which sets the Baud rate.
  - Some more (details, as always, in the data sheet).

# UCSRnA

**UCSRnA – USART Control and Status Register A**

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| | RXCn | TXCn | UDREn | FEn | DORn | UPEn | U2Xn | MPCMn | UCSRnA |
| Read/Write | R | R/W | R | R | R | R | R/W | R/W | |
| Initial Value | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | |

- **Bit 7 – RXCn: USART Receive Complete**

This flag bit is set when there are unread data in the receive buffer and cleared when the receive buffer is empty (i.e., does not contain any unread data). If the Receiver is disabled, the receive buffer will be flushed and consequently the RXCn bit will become zero. The RXCn Flag can be used to generate a Receive Complete interrupt (see description of the RXCIEn bit).

- **Bit 6 – TXCn: USART Transmit Complete**

This flag bit is set when the entire frame in the Transmit Shift Register has been shifted out and there are no new data currently present in the transmit buffer (UDRn). The TXCn Flag bit is automatically cleared when a transmit complete interrupt is executed, or it can be cleared by writing a one to its bit location. The TXCn Flag can generate a Transmit Complete interrupt (see description of the TXCIEn bit).

- **Bit 5 – UDREn: USART Data Register Empty**

The UDREn Flag indicates if the transmit buffer (UDRn) is ready to receive new data. If UDREn is one, the buffer is empty, and therefore ready to be written. The UDREn Flag can generate a Data Register Empty interrupt (see description of the UDRIEn bit).

# Example Receive Code

```
// Receive a character from the USART.
// Blocks until a character is received.

uint8_t usart1_getc(void) // Returns a byte, takes no arguments.
{
    // Wait for something in the receive register.
    // Checks bit Receive Complete (RXC).
    while ((UCSR1A & (1 << RXC1)) == 0) { // RXC1 == 7.
        continue;
    }

    // Bit 7 of UCSR1A (RXC1) is now == 1
    // Data is present to be read!

    return UDR1;
}
```

# Example Send Code

```
// Transmit a character from the USART.
// Blocks until the character can be sent.

void usart1_putc(char c) // Takes a character, returns nothing.
{
    // Wait for the transmit register to be empty.
    // Checks bit UART Data Register Empty (UDRE)
    while ((UCSR1A & (1 << UDRE1)) == 0) { // UDRE1 == 5.
        continue;
    }

    // Bit 5 of UCSR1A (UDRE1) is now == 1
    // Buffer is empty and can be written to!

    // Write to data register so the hardware can send the new byte.
    UDR1 = c;
}
```

# Fun Kit `ir_uart.h` Library Sending

```
// Located in drivers/avr/
#include "ir_uart.h"

// Other libraries...

void main(void)
{
    system_init();
    ir_uart_init(); // Initialise the UART.

    // Send a character at maximum speed.
    while (true) {
        ir_uart_putc('?'); // Send a character.
        // Blocks until transfer can be started.
    }
}
```

# Fun Kit `ir_uart.h` Library Receiving

```c
#include "ir_uart.h"

// Other libraries...

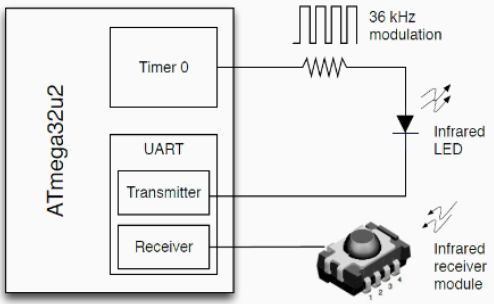
void main(void)
{
    system_init();
    ir_uart_init(); // Initialise the UART.

    // Actively listen for UART data.
    while (true) {
        if (ir_uart_read_ready_p()) { // Are we ready to receive data?
            char c;
            c = ir_uart_getc(); // Get character from data register UDR1.

            // Process the received byte.
        }

        // Can do other tasks while we wait for UART data.
    }
}
```

# Infra-Red Communications Modulation




- The IR LED is modulated when on to distinguish the signal from ambient IR.
- Timer 0 (the 8-bit `TCNT0`) is used to modulate the sent IR at $36$ kHz.
- The receiver demodulates the IR, outputting a low when there is light, and high when there is no light. The sending is therefore inverted.

# Key Concepts

- ICs can communicate a single bit at a time (serially) by e.g. changing the voltage on a wire.
- There are different ways to connect serial communications channels to allow for two-way communication (duplex).
- UART is a type of serial communications, used between ICs and boards.
- UART has options that both the sender and receiver both need to know.
- The ATmega32U2 has a dedicated peripheral for handling UART.
- The Fun Kits send their UART over modulated IR to other boards.

# Homework

One UART peripheral is communicating over a wire to another. The UARTs have been configured for 7-bit data, the LSB is transmitted first, and the idle state is high. There is one start bit, two stop bits, and an even parity bit.

Sketch the waveform that should be sent on the wire to transmit the ASCII character `'A'`?